

Meridian TCE

Pre-Deployment Review: Frontend & Backend Shortcomings

Prepared for VPS self-hosting

Date: 2 June 2026

Scope: Express + SQLite backend (server/) and Vite/React frontend (src/)

Severity summary

- CRITICAL** 3 — Fix before exposing the server to the internet
- HIGH** 6 — Fix before real users / real data
- MEDIUM** 7 — Should fix soon after launch
- LOW** 5 — Quality / hardening / cleanup

This report lists concrete, fixable shortcomings found by reading the source. Items are ordered by severity. The three CRITICAL items are exploitable as written and should be resolved before the app is reachable from the public internet.

1. Backend (server/)

CRITICAL B1. A live Anthropic API key is stored in server/.env

Where: server/.env (ANTHROPIC_API_KEY=sk-ant-api03-...).

Problem: A real, working API key is sitting in a plaintext file in the project. Anyone who gets read access to the repo, a backup, a misconfigured deploy, or the VPS filesystem can bill your Anthropic account without limit. Because this file was shared, the key must be treated as already compromised.

Fix: Rotate (revoke + reissue) this key immediately in the Anthropic console. Never commit a real .env. On the VPS, create .env directly on the box with permissions 600, owned by the service user. Confirm .env is gitignored (it is) before the project ever becomes a git repo.

CRITICAL B2. No per-user authorization on entities (broken access control / IDOR)

Where: server/index.js entity routes; server/db.js filterEntities/getEntity/updateEntity.

Problem: Every entity route only checks requireAuth, never ownership. The created_by filter is applied client-side only. Any logged-in user can: (a) POST /api/entities/ConsultationResult/filter with an empty query and read ALL clients' consultation results, org dossiers, and consultant twins; (b) GET /api/entities/<type>/<id> for any id; (c) PUT /api/entities/<type>/<id> to overwrite anyone's record. This exposes confidential client data across all accounts.

Fix: Enforce ownership on the server. For non-admins, force query.created_by = req.user.email in filter, and verify rec.created_by === req.user.email (or role admin) before returning/updating in get/update. Do not rely on the frontend to scope data.

CRITICAL B3. Default/placeholder JWT secret will be deployed

Where: server/auth.js (JWT_SECRET fallback) and server/.env.example / .env.

Problem: JWT_SECRET falls back to "change-me-in-production", and the shipped .env contains "some-long-random-string-you-make-up" — a guessable placeholder, not a secret. If deployed as-is, anyone can forge a JWT (e.g. role:"admin") and take over the app. Tokens also last 30 days with no revocation.

Fix: Generate a real secret (e.g. `openssl rand -hex 32`) and set it in the VPS .env. Make the server refuse to start if JWT_SECRET is unset or equals the placeholder, rather than only warning. Consider a shorter TTL plus refresh.

HIGH B4. CORS is fully open (Access-Control-Allow-Origin: *)

Where: server/index.js — app.use(cors()).

Problem: cors() with no options allows any website to call your API from a browser. Combined with token-in-localStorage, this widens CSRF/abuse surface and lets any origin hit your AI endpoints (which cost money).

Fix: Restrict to your own origin: cors({ origin: "https://your-domain.com" }). Since the Express server also serves the SPA in production, you can disable CORS entirely for same-origin.

HIGH B5. No rate limiting on auth or AI endpoints

Where: All routes, especially /api/auth/login, /api/auth/register, and /api/functions/* (Anthropic calls).

Problem: Login has no throttling! unlimited password brute-force. Registration accounts. The AI function routes call the paid Anthropic API with up to 8000 max_tokens and no per-user quota! a single authenticated user can run up a large bill or exhaust

Fix: Add express-rate-limit (strict on login/register, modest on AI routes). Add a per-user daily token/usage cap. Consider gating registration after the first admin (see B6).

HIGH B6. Open self-registration; weak/insecure password handling

Where: server/index.js /api/auth/register; verifyPassword in auth.js.

Problem: Anyone who can reach the server can create an account (role "user") and immediately gain access to all entity routes (compounded by B2). There is no email verification, no password strength rule (a 1-char password is accepted), and no lockout. The "first account becomes admin" rule means if you don't register first on a fresh deploy, an attacker can claim admin.

Fix: Disable open registration or put it behind an invite/allowlist after the first admin exists. Enforce a minimum password policy server-side. Register the admin account immediately on first boot.

HIGH B7. No security headers; SPA fallback + static files unhardened

Where: server/index.js — no helmet; express.static for /uploads and dist.

Problem: No Content-Security-Policy, X-Content-Type-Options, HSTS, X-Frame-Options, etc. Uploaded files are served publicly from /uploads with their original content; a malicious upload (e.g. an HTML/SVG file) served same-origin can enable stored XSS against logged-in users.

Fix: Add helmet(). Serve uploads with Content-Disposition: attachment and a restrictive Content-Type, or behind auth. Restrict accepted upload types to the ones you actually parse (pdf/docx/txt/csv/md/json).

HIGH B8. Unbounded full-table scan + JSON.parse on every filter call

Where: server/db.js filterEntities — SELECT * then map/JSON.parse/filter in JS.

Problem: Filtering loads every row of a type into memory, JSON.parses each, then filters in JavaScript. With growth this is slow and memory-heavy, and there is no default limit — a filter with no limit returns everything. Combined with B2/B5 it is also a cheap denial-of-service vector.

Fix: Push created_by/type filtering into SQL (you already index both columns). Apply a sane default and maximum limit server-side. For frequently queried JSON fields, store them as real columns.

MEDIUM B9. AI JSON parsing is fragile (regex brace match)

Where: server/ai.js onboardingChat and extractFromText — raw.match(/\[{\s\S}*\}/).

Problem: Model output is parsed by grabbing the first "{" to the last "}". Any prose around or within (nested braces, code fences, multiple objects) breaks parsing, silently falling back to a 50% calibration stub or dumping raw text. Users see degraded results with no error.

Fix: Use the model's structured/tool-output mode or request strict JSON, validate against the schema (zod), and surface a real error/retry instead of a silent fallback.

MEDIUM B10. response.content[0].text assumed to always exist

Where: server/ai.js — every handler reads response.content[0].text directly.

Problem: If the API returns a non-text first block, an empty content array, a refusal, or a stop_reason of max_tokens, content[0].text can be undefined and throw, returning a 500 with a stack to the client.

Fix: Defensively read the first text block; handle empty/refusal responses; return a friendly error.

MEDIUM B11. Errors returned to client include raw messages; no structured logging

Where: server/index.js error middleware — res.json({ error: err.message }).

Problem: Internal error messages (including potential file paths or DB details) are sent to the client. There is also no real logging/monitoring, so you will not notice abuse or failures on the VPS.

Fix: Return a generic message for 500s, log details server-side only (with a request id), and add a logger (pino/morgan) plus PM2 log rotation.

MEDIUM B12. Uploaded files are never cleaned up; disk can fill

Where: server/index.js upload handler; UPLOAD_DIR.

Problem: Every upload (up to 25 MB) is stored forever. There is no retention/cleanup and no link between a file and the entity that used it, so a VPS with a small disk will eventually fill, taking the app (and SQLite WAL) down.

Fix: Add a retention/cleanup job or delete after extraction; cap total storage; monitor disk usage.

MEDIUM B13. No request validation on entity type / body shape

Where: server/index.js — req.params.type and req.body passed straight through.

Problem: Any string is accepted as an entity "type", so clients can create arbitrary collections. Bodies are stored as-is (only managed columns stripped), so there is no schema enforcement and inconsistent/garbage data can accumulate.

Fix: Allowlist the known entity types and validate payloads with zod per type.

2. Frontend (src/)

HIGH F1. Auth token stored in localStorage (XSS-exploitable)

Where: src/api/base44Client.js — getToken/setToken use localStorage.

Problem: The 30-day JWT lives in localStorage, readable by any JavaScript on the page. Any XSS (made more likely by B7 unrestricted uploads and the react-quill rich text) leaks a long-lived token that grants full account access. localStorage tokens are not cleared when the browser/tab closes either.

Fix: Prefer an httpOnly, Secure, SameSite cookie set by the server. If you must keep localStorage, shorten TTL and add CSP + strict upload handling to reduce XSS risk.

HIGH F2. Not responsive — viewport hard-locked to 1440px

Where: index.html — <meta name="viewport" content="width=1440">.

Problem: The viewport is pinned to a fixed 1440px width, so on phones and smaller laptops the whole app is zoomed/scaled and effectively unusable. For a public-facing self-hosted product this is a serious usability gap.

Fix: Use the standard <meta name="viewport" content="width=device-width, initial-scale=1"> and make the layout responsive (the heavy inline fixed-pixel styling will need attention).

MEDIUM F3. Dead/duplicate auth code: Supabase + duplicate Login

Where: src/lib/SupabaseAuthContext.jsx and src/components/auth/Login.jsx (alongside the real src/pages/Login.jsx and src/lib/AuthContext.jsx).

Problem: Leftover Supabase auth context and a second Login component remain from a previous stack. Dead auth code is confusing and risky — it is easy to wire up the wrong one, and it bloats the bundle. base44Client.js naming also still pretends to be a third-party SDK.

Fix: Delete the unused SupabaseAuthContext and the duplicate Login. Rename base44Client.js to something honest (e.g. apiClient.js).

MEDIUM F4. Giant 1,350-line page component (MeridianTCE.jsx)

Where: src/pages/MeridianTCE.jsx (1,350 lines).

Problem: A single file holds routing, state, an event-bus class, markdown/table parsing, entity access, and rendering. This is hard to test, hard to change safely, and re-renders broadly. It also mixes data-access logic that should enforce ownership but cannot (see B2).

Fix: Split into hooks (data fetching), small components, and pure helper modules. Move shared parsing utilities out of the page.

MEDIUM F5. No global error boundary or network-error UX

Where: src/App.jsx, src/main.jsx.

Problem: There is no React error boundary, so a render error white-screens the whole app. API failures throw generic Error objects; many call sites surface raw messages or nothing. On a flaky VPS connection the UX degrades poorly.

Fix: Add an error boundary at the app root and consistent toast-based handling for API errors (react-hot-toast/sonner are already installed).

MEDIUM F6. Client/role model is inconsistent (user vs client)

Where: Backend roles "admin"/"user"/"client"; frontend ClientView and ProtectedRoute logic.

Problem: register creates role "user", setClientRole sets "client", requireAdmin checks "admin". The three roles are not consistently defined or enforced, and ProtectedRoute/ClientView gate on role-ish state that the backend does not authoritatively check per resource. This is a correctness and security smell (ties back to B2).

Fix: Define the role set explicitly, document what each can do, and enforce it server-side on every route.

LOW F7. No tests anywhere in the project

Where: Entire repo (no *.test.* outside node_modules).

Problem: There is no automated test for auth, ownership, entity CRUD, or AI parsing. Regressions (especially around the access-control fixes above) will be easy to introduce and hard to catch.

Fix: Add at least a handful of API tests (auth, ownership enforcement, filter scoping) with vitest/supertest.

LOW F8. Heavy dependencies likely unused / large bundle

Where: package.json (three.js, react-leaflet, embla, recharts, framer-motion, moment + date-fns, etc.).

Problem: The frontend pulls in several large libraries (3D, maps, carousels, charts, two date libraries). If unused they bloat the bundle and slow first load on a modest VPS. moment is also legacy/deprecated alongside date-fns.

Fix: Audit with the build output / a bundle analyzer; drop unused deps; standardize on date-fns and remove moment.

LOW F9. Fixed viewport meta also harms accessibility/SEO

Where: index.html.

Problem: Beyond responsiveness (F2), the fixed width prevents user zoom on mobile and hurts Lighthouse/SEO scores — relevant for a public deployment.

Fix: Covered by F2; ensure zoom is not disabled.

3. Deployment & operations (VPS)

HIGH D1. Server binds and is intended to be hit directly on :8787 over HTTP

Where: README run instructions; `app.listen(PORT)`.

Problem: The README's first-pass instructions expose `http://<vps-ip>:8787` directly — no TLS, tokens and passwords sent in cleartext. The Nginx+certbot step is presented as optional ("recommended").

Fix: Always terminate TLS (Nginx + certbot, as in the README's later section). Bind Node to 127.0.0.1 and only expose 443 via the reverse proxy. Add a firewall (ufw) blocking 8787 from the outside.

MEDIUM D2. No process limits, no backups automation, SQLite single-file risk

Where: `server/data/meridian.db` (WAL), `server/uploads`.

Problem: All data lives in one SQLite file plus an uploads dir. The README mentions backing these up manually but nothing is automated. A VPS disk failure or a bad deploy can lose everything. PM2 is suggested but there is no memory cap or restart policy tuning.

Fix: Automate a nightly backup of the DB (use the SQLite online-backup/.backup, not a raw copy of WAL) and uploads off-box. Set PM2 max-memory-restart and log rotation.

LOW D3. No environment validation on boot

Where: server startup.

Problem: The server starts even with a missing/placeholder `JWT_SECRET` (warn only) and only fails on the first AI call if `ANTHROPIC_API_KEY` is missing. Misconfiguration surfaces late, in front of users.

Fix: Validate required env (`JWT_SECRET` strength, `ANTHROPIC_API_KEY` presence) at startup and exit non-zero if invalid.

4. Recommended fix order

Before the server is reachable from the internet

- Rotate the leaked Anthropic key and set a real JWT_SECRET (B1, B3).
- Enforce per-user ownership on every entity route (B2).
- Put it behind Nginx + HTTPS, bind Node to localhost, firewall 8787 (D1).

Before real users / real data

- Lock down CORS, add rate limiting, restrict registration, add helmet & safe upload serving (B4–B7).
- Move the token to an httpOnly cookie or tightly scope XSS risk (F1).
- Make the app responsive (F2).

Soon after launch

- Harden AI JSON parsing and API response handling; generic error responses + logging (B9–B11).
- SQL-side filtering with limits; upload retention; entity-type validation (B8, B12, B13).
- Automated backups, env validation, PM2 limits (D2, D3).

Quality / cleanup

- Remove dead Supabase/duplicate-login code; split MeridianTCE.jsx; add tests; trim dependencies (F3–F8).

Note: this review is based on reading the source, not on a running penetration test. The CRITICAL items (B1-B3) are confirmed by the code and the contents of server/.env and should be treated as urgent.